Assignment 12

Name:Vanga Ranjith
Hallticket No:2303A51734

Hyperledger Fabric Problem Solving – Personal Asset Portfolio

1 . Objective

To design and implement a simple Hyperledger Fabric smart contract (chaincode) that manages a personal asset portfolio on a blockchain network. The practical focuses on problem-solving using Fabric concepts such as assets, transactions, ledger state, and queries.

2 . Problem Statement

Implement a basic blockchain-based portfolio system where a user can:

- Create assets (e.g., certificates, projects, or tokens)
- View all assets stored on the blockchain
- Transfer asset ownership between users

This practical demonstrates how Hyperledger Fabric solves real-world problems like data immutability, transparency, and secure ownership transfer.

3 . Requirements

OS	Windows / Linux / macOS
Node.js	v16 or later
Docker	& Docker Compose
Git	For cloning fabric - samples
VS Code	With Docker + JavaScript extensions
Fabric Platform	Hyperledger Fabric test network

4 . Chaincode Design – portfolioContract.js

The chaincode is written in JavaScript (Node.js) and implements the **Contract** class from the *fabriccontractapi* package.

Step 1 – Initialise the chaincode project

```
mkdir portfolio-chaincode &&cdportfolio-chaincode npm
init -y npm install fabric-contract-api fabricshim
```

Step 2 – portfolioContract.js (full chaincode)

```
"use strict"; const { Contract}=
require("fabriccontract-api"); class
PortfolioContract extends Contract{

// ■■ Initialise ledger with sample assets
■■■
```

```
owner: "Karthik", value: 15000, createdAt: "2025-02-20" }, { id:
"TKN001", name: "DeFi Token Alpha", type: "Token", owner: "Karthik",
value: 8000, createdAt: "2025-03-05" }, ]; for (const asset of assets)
{ await ctx.stub.putState(asset.id,
Buffer.from(JSON.stringify(asset))); console.log(`Initialised asset
${asset.id}`); } console.log("Ledger initialised with", assets.length,
"assets."); }

// ▬▬ Create a new asset
▪▪▪
```

```
const previousOwner = asset.owner; asset.owner
= newOwner;

await ctx.stub.putState(id, Buffer.from(JSON.stringify(asset)));
console.log(`Asset ${id} transferred: ${previousOwner} -> ${newOwner}`); return
previousOwner; // return previous owner }


// ■■ Check existence
■■■
```

Step 4 – package.json main entry

```
{ "name": "portfoliochaincode",
"version": "1.0.0", "main":
"index.js",
"scripts": { "start": "fabricchaincode-node
start"
},
"dependencies": {
"fabric-contract-api": "^2.5.0", "fabricshim":
"^2.5.0"
} }
```

Step5–Deploychaincodeontestnetwork

```
cd ../fabric-samples/test-network ./network.sh up createChannel -c

portfoliochannel -ca

./network.sh deployCC \
-ccn portfolio \
-ccp ../../portfolio-chaincode \
```

```
-ccl javascript \ -c portfoliochannel Deploying chaincode
'portfolio' on channel 'portfoliochannel'...
```

```
Chaincode is packaged.
Installing chaincode on peer0.org1... Done.
Installing chaincode on peer0.org2... Done.
Chaincode definition committed on channel
'portfoliochannel'. Committed chaincode definition for
chaincode 'portfolio':Version: 1.0 Sequence: 1
```

Step 6 – Initialise ledger

```
peer chaincode invoke \ -o localhost:7050 --
ordererTLSHostnameOverride orderer.example.com \
--tls --cafile "$ORDERER_CA" \
-C portfoliochannel -n portfolio \
--peerAddresses localhost:7051 --tlsRootCertFiles "$ORG1_CA" \ -
peerAddresses localhost:9051 --tlsRootCertFiles "$ORG2_CA" \
-c '{"function":"InitLedger","Args":[]}'
```

```
Chaincode invoke successful. result: status:200
Initialised asset CERT001 Initialised asset
PROJ001
Initialised asset TKN001 Ledger initialised
with 3 assets.
```

Step 7 – Create a new asset

```
peer chaincode invoke \ -o localhost:7050 -ordererTLSHostnameOverride
orderer.example.com \
--tls --cafile "$ORDERER_CA" \
-C portfoliochannel -n portfolio \
--peerAddresses localhost:7051 --tlsRootCertFiles "$ORG1_CA" \
--peerAddresses localhost:9051 --tlsRootCertFiles "$ORG2_CA" \
-c '{"function":"CreateAsset","Args":["CERT002","Google Cloud
ACE","Certificate","Karthik","7000"]}'
```

```
Chaincode invoke successful. result:
status:200 payload: {
"id": "CERT002",
"name": "Google Cloud ACE",
"type": "Certificate",
"owner": "Karthik",
"value": 7000, "createdAt": "2025-04-
12T10:45:32.000Z"
}
```

Step8–Queryallassets

```
peer chaincode query -C portfoliochannel -n portfolio \ -
c '{"Args":["GetAllAssets"]}'
```

```
[{
"id":"CERT001","name":"AWS Cloud Practitioner","type":"Certificate",
"owner":"Karthik","value":5000,"createdAt":"2025-01-10"
},
{
"id":"CERT002","name":"Google Cloud ACE","type":"Certificate",

"owner":"Karthik","value":7000,"createdAt":"2025-04-12T10:45:32.000Z"
```

```
},
{
"id":"PROJ001","name":"E-Commerce Platform","type":"Project",
"owner":"Karthik","value":15000,"createdAt":"2025-02-20"
},
{
"id":"TKN001","name":"DeFi Token Alpha","type":"Token",
"owner":"Karthik","value":8000,"createdAt":"2025-03-05"
} ]
```

Step9–Transferassetownership

```
peer chaincode invoke \ -o localhost:7050 --
ordererTLSHostnameOverride orderer.example.com \
--tls --cafile "$ORDERER_CA" \
-C portfoliochannel -n portfolio \
--peerAddresses localhost:7051 --tlsRootCertFiles "$ORG1_CA" \ --peerAddresses
localhost:9051 --tlsRootCertFiles "$ORG2_CA" \ -c
'{"function":"TransferAsset","Args":["CERT002","RajeshKumar"]}'
```

```
Chaincode invoke successful. result: status:200 payload: "Karthik"

Asset CERT002 transferred: Karthik -> RajeshKumar
```

Step10–Verifytransfer(readsingleasset)

```
peer chaincode query -C portfoliochannel -n portfolio \ -
c '{"Args":["ReadAsset","CERT002"]}'
```

```
{
"id": "CERT002",
"name": "Google Cloud ACE",
"type": "Certificate",
"owner": "RajeshKumar",
"value": 7000, "createdAt": "2025 - 04 -1 2 T10:45:32.000Z"
}
```

5.Architecture Overview

```
Fabric Test Network
Channel: portfoliochannel
Org1 Peer                                Org2 Peer
(port 7051)                              (port 9051)
```

```
Orderer ( Raft consensus, port 7050)
Chaincode: PortfolioContract (JS) deployed on both peers
Ledger: LevelDB (default state DB)

Client CLI peer chaincode invoke/query Endorsing Peers
Orderer (ordering + batching)
Committing Peers (ledger update)
```

6 . Result & Conclusion

The **PortfolioContract** chaincode was successfully deployed on a 2-org Hyperledger Fabric test network on channel *portfoliochannel*. The followingoperations were demonstrated:

Function	Description	Status ✓
InitLedger	3 sample assets initialised on-chain	status:200 ✓
CreateAsset	New certificate asset CERT002 created	status:200 ✓
GetAllAssets	All 4 assets retrieved as JSON array	status:200 ✓
TransferAsset	CERT002 transferred Karthik→RajeshKumar	status:200 ✓
ReadAsset	CERT002 owner confirmed as RajeshKumar	status:200

The practical demonstrated Hyperledger Fabric's ability to solve real-world problems: data immutability ensures assets cannot be altered without a traceable transaction; channel isolation provides privacy; and the chaincode logic enforces business rules such as preventing duplicate asset IDs. Fabric's permissioned architecture makes it ideal for enterprise portfolio, supply-chain, and credential management systems.